

SKILL WEEK 3

SESSION 5

Task 1: TO Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50

struct Token {
    char value[MAX_INPUT];
};

int main() {
    char input[MAX_INPUT];
    struct Token tokens[MAX_TOKENS];
    int count = 0;
    int in_quotes = 0;
    int j = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty input.\n");
        return 0;
    }

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\' ) {
            in_quotes = !in_quotes;
            continue;
        }

        if ((input[i] == ' ' || input[i] == '\t') && !in_quotes) {
            if (j > 0) {
                tokens[count].value[j] = '\0';
            }
        } else {
            tokens[count].value[j++] = input[i];
        }

        if (j > 0) {
            tokens[count].value[j] = '\0';
            count++;
            j = 0;
        }
    }

    if (in_quotes) {
        printf("Syntax Error: Unclosed single quote.\n");
        return 1;
    }

    printf("\nParsed Tokens:\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: [%s]\n", i + 1, tokens[i].value);
    }

    printf("\nParsing completed successfully.\n");

    return 0;
}
```

```
Argument 1: 'hello world'
ruhi0404@Ruhitha:~/Operating_Systems$ nano skills5task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills5task1.c -o skills5task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills5task1
Enter command: hello world

Parsed Tokens:
Token 1: [hello]
Token 2: [world]

Parsing completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully handled single-quoted strings by preserving their literal content and treating spaces inside quotes as part of the same token. Variable-like text inside single quotes was preserved without expansion. The program also detected unclosed single quotes and displayed an appropriate syntax error, demonstrating basic quote-aware parsing and validation.

Task 2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKEN 50
#define MAX_LENGTH 100

struct Token {
    char value[MAX_LENGTH];
};

int main() {
    char input[MAX_INPUT];
    struct Token tokens[MAX_TOKEN];
    int count = 0;
    int in_quotes = 0;
    int j = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    for (int i = 0; input[i] != '\0'; i++) {
        /* Handle double quotes */
        if (input[i] == '"') {
            in_quotes = !in_quotes;
            continue;
        }
    }

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]W Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T E
[^]J J

```

        in_quotes = !in_quotes;
        continue;
    }

    /* Handle variable expansion */
    if (input[i] == '$' && in_quotes) {
        char variable[50];
        int k = 0;
        i++;

        while (input[i] != '\0' &&
            (input[i] >= 'A' && input[i] <= 'Z' ||
             input[i] >= 'a' && input[i] <= 'z' ||
             input[i] == '_')) {
            variable[k++] = input[i];
            i++;
        }

        variable[k] = '\0';
        i--;

        char *value = getenv(variable);

        if (value != NULL) {
            for (int x = 0; value[x] != '\0'; x++) {
                tokens[count].value[j++] = value[x];
            }
        }

        continue;
    }

    /* Preserve spaces inside double quotes */
    if ((input[i] == ' ' || input[i] == '\t') && !in_quotes) {
        if (j > 0) {
            tokens[count].value[j] = '\0';
            count++;
        }
    }

```

```

        continue;
    }

    /* Preserve spaces inside double quotes */
    if ((input[i] == ' ' || input[i] == '\\t') && !in_quotes) {
        if (j > 0) {
            tokens[count].value[j] = '\\0';
            count++;
            j = 0;
        }
    } else {
        tokens[count].value[j++] = input[i];
    }
}

/* Detect unclosed quotes */
if (in_quotes) {
    printf("Syntax Error: Unclosed double quote.\\n");
    return 1;
}

if (j > 0) {
    tokens[count].value[j] = '\\0';
    count++;
}

printf("\\nParsed Tokens:\\n");

for (int i = 0; i < count; i++) {
    printf("Token %d: [%s]\\n", i + 1, tokens[i].value);
}

printf("\\nParsing completed successfully.\\n");

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skills5task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills5task2.c -o skills5task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills5task2
Enter command: hello world

Parsed Tokens:
Token 1: [hello]
Token 2: [world]

Parsing completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills5task2
Enter command: "hello world"

Parsed Tokens:
Token 1: [hello world]

Parsing completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills5task2
Enter command: echo "HELLLO WOR"

Parsed Tokens:
Token 1: [echo]
Token 2: [HELLLO WOR]

Parsing completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully handled double-quoted strings while preserving spaces within the quoted content. Variable expansion was performed for environment variables such as \$HOME. The program also detected unclosed double quotes and displayed an appropriate syntax error. Thus, the experiment demonstrated quote-aware parsing, variable expansion, token generation, and validation of quoted commands.

SESSION 6

Task 1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_LENGTH 100

struct Token {
    char value[MAX_LENGTH];
};

int main() {
    char input[MAX_INPUT];
    struct Token tokens[MAX_TOKENS];
    int count = 0;
    int j = 0;
    int escaped = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    for (int i = 0; input[i] != '\0'; i++) {
        /* Handle escape sequence */
        if (escaped) {
            tokens[count].value[j++] = input[i];
            escaped = 0;
            continue;
        }
    }
}
```

 Help	 Write Out	 Where Is	 Cut
 Exit	 Read File	 Replace	 Paste

```

        escaped = 1;
        continue;
    }

    /* Space separates tokens */
    if (input[i] == ' ' || input[i] == '\t') {
        if (j > 0) {
            tokens[count].value[j] = '\0';
            count++;
            j = 0;
        }
    } else {
        tokens[count].value[j++] = input[i];
    }
}

/* Handle trailing escape character */
if (escaped) {
    tokens[count].value[j++] = '\\';
}

if (j > 0) {
    tokens[count].value[j] = '\0';
    count++;
}

printf("\nParsed Tokens:\n");

for (int i = 0; i < count; i++) {
    printf("Token %d: [%s]\n", i + 1, tokens[i].value);
}

printf("\nParser output validated successfully.\n");

return 0;

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skills6task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills6task1.c -o skills6task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills6task1
Enter command: echo hello\ world

Parsed Tokens:
Token 1: [echo]
Token 2: [hello world]

Parser output validated successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully handled escape sequences using the backslash character. Escaped spaces were preserved as part of the same token, while special symbols were treated as literal characters. The parser correctly separated normal tokens and preserved escaped characters, demonstrating proper handling of complex input and validation of parser output.

Task 2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];
    char argument[100];
    pid_t pid;
    int status;

    printf("Enter command: ");
    scanf("%s", command);

    printf("Enter argument: ");
    scanf("%s", argument);

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Executing: %s %s\n", command, argument);

        execlp(command, command, argument, NULL);

        perror("Execution failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
    }
}
```

```
GNU nano 7.2

printf("Enter argument: ");
scanf("%s", argument);

pid = fork();

if (pid < 0) {
    perror("Fork failed");
    return 1;
}

if (pid == 0) {
    printf("\nChild Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Executing: %s %s\n", command, argument);

    execlp(command, command, argument, NULL);

    perror("Execution failed");
    exit(1);
}
else {
    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
        printf("Child process completed successfully.\n");
    else
        printf("Child process terminated abnormally.\n");
}

return 0;
}
```

```
ruhi0404@Ruhitha:~/Operating_Systems$ nano skills6task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills6task2.c -o skills6tas
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills6task2
Enter command: ls
Enter argument: -1

Parent Process
Parent PID: 889
Child PID: 890

Child Process
Child PID: 890
Executing: ls -l
File1.txt
File2.txt
LICENSE
Makefile
README.md
assets
bin
docs
include
obj
pract1task1
pract1task1.c
pract2task2
pract3task1
pract3task1.c
pract3task2
pract3task2.c
pract1task1
pract1task1.c
pract2task1
pract2task1.c
practical
sample.txt
scripts
skill
skill1task2
skill1task2.c
skill2task1
```

```
pra3task2.c
prac1task1
prac1task1.c
prac2task1
prac2task1.c
practical
sample.txt
scripts
skill
skill1task2
skill1task2.c
skill2task1
skill2task1.c
skill2task2
skill2task2.c
skill2w1task1
skill2w1task1.c
skill2w1task2
skill2w1task2.c
skills3task2
skills3task2.c
skills3week2
skills3week2.c
skills4task1
skills4task1.c
skills4task2
skills4task2.c
skills5task1
skills5task1.c
skills5task2
skills5task2.c
skills6task1
skills6task1.c
skills6task2
skills6task2.c
skillw3task2
skillw3task2.c
src
tests
Child process completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully created a child process using `fork()` and executed a Linux command in the child process using `execlp()`. Command arguments were passed to the child process, while the parent process waited for the child to complete using `waitpid()`. Execution errors were handled using `perror()`. Thus, the experiment demonstrated process creation, program execution, argument passing, parent-child process management, and command launching in Linux.